Lezione 6 · Una variabile per non perdersi nei numeri

Giovedì 29 ottobre 2026 · 90 minuti · 48 minuti di tastiera

Cosa fa questa lezione

Il problema vero, visto in classe: scrivono quello che devono fare, mettono anche qualche commento, e poi **si perdono nei numeri**. `forward(100)`, la casetta, la finestra, la porta — e per spostare o ingrandire qualcosa devono rimettere mano a quindici righe e sbagliano.

Oggi arrivano le due cose che tolgono quella fatica, in quest'ordine:

		Toglie
La variabile	un nome in cima, e tutto si calcola da lì	«mi perdo nei numeri» e «non riesco a posizionarlo»
Il ciclo	scrivi una volta, succede otto volte	«ho scritto cinque volte la stessa cosa»

La variabile prima, e con più tempo. È quella che risolve il problema che hanno adesso. Il ciclo arriva nella seconda metà, con lo steccato di un loro compagno come esempio — e si approfondisce la settimana dopo.

> **Le tre variabili che cambiano la vita**
>
> ```
> groesse = 120      # quanto è grande
> start_x = -60      # dove sta, destra/sinistra
> start_y = -120     # dove sta, su/giù
> ```
>
> Con queste tre righe in cima, il disegno si ingrandisce e si sposta cambiando un numero solo. Il «non riesco a orientare il disegno» sparisce.

Il materiale

Cosa	Quanto	Nota
Portatili con Thonny	11 + 1	
`haus_variabel.py`, `treppe_kurz.py`, `zaun_kurz.py`, `vieleck.py`	—	Nella cartella `python_kurs`, copiati il mercoledì
`fehler/fehler_7.py` e `fehler_8.py`	—	Nella sottocartella `fehler`, accanto ai sei di giovedì
Il file dello steccato di un ragazzo	1	Quello fatto giovedì. Chiedigli il permesso in pubblico
Il foglio per chi va forte	1–2 copie	Le quattro sfide matematiche, in fondo a questo documento
Proiettore, quaderni, foglietti	—	Come sempre

I novanta minuti

Minuti	Fase	
0–5	Apertura	I loro disegni di giovedì sul proiettore
5–20	La variabile	`haus_variabel.py`: due numeri, due stupori
20–45	Officina 1	Ognuno mette le variabili **nel proprio** disegno
45–60	Il ciclo	Lo steccato di un compagno, poi `vieleck.py`. E i due errori nuovi
60–83	Officina 2	Il ciclo nel proprio disegno, o finire le

		variabili
83–90	Conta e quaderno	Righe prima, righe dopo

Fase 1 · Apertura (0–5)

Proietti due o tre dei loro disegni di giovedì, così come sono. Niente commenti sui difetti.

Poi la frase che apre:

> »Heute lernt ihr zwei Tricks. Mit dem ersten müsst ihr nicht mehr fünfzehn Zahlen ändern, sondern eine. Mit dem zweiten müsst ihr nicht mehr fünf Mal das Gleiche tippen.«

Fase 2 · La variabile (5–20)

Primo stupore: la grandezza

Apri `haus_variabel.py` al proiettore e lo esegui. Esce una casa con porta e finestra.

Poi, davanti a loro, cambi **un numero solo**:

```
groesse = 120      →      groesse = 220
```

Play. La casa è grande il doppio: muri, tetto, porta, finestra, tutto insieme, tutto proporzionato.

> »Eine Zahl. Nicht fünfzehn.«

Secondo stupore: la posizione

Adesso cambi gli altri due:

```
start_x = -60      →      start_x = 150
start_y = -120     →      start_y = 50
```

Play. La casa è da un’altra parte. Nient’altro è cambiato.

Questo è il momento che risolve il problema che hanno: **non si sposta un disegno spostando quindici righe, si sposta cambiando dove comincia.**

Come funziona, in tre frasi

Non più di tre, e nessuna definizione.

1. `groesse = 120` vuol dire: **da adesso in poi, dove scrivo `groesse`, c’è scritto 120.**
2. Deve stare **in cima**, prima di essere usata. Se la scrivi dopo, Python non la conosce ancora.
3. Si può anche calcolare: `groesse * 0.5` è la metà, `groesse * 2` il doppio. L’asterisco è sul foglietto, Umschalt + il tasto più.

> **La scatola con l’etichetta, se vuoi un minuto di fisico**
>
> Una scatola vera sul tavolo, un’etichetta `groesse` sopra, e dentro un foglietto col 120. Tiri fuori il foglietto, ne metti uno col 220, e la scatola si chiama ancora `groesse`.
>
> Sessanta secondi, non di più. Con questo gruppo il tempo sta meglio sulla tastiera, ma il nome «scatola» torna utile tutto l’anno quando qualcuno chiede cos’è una variabile.

La domanda da fare prima di mandarli al computer

> »Was passiert, wenn ich `groesse` ändere, NACHDEM das Haus schon gezeichnet ist?«

Risposta: niente. La variabile vale da dove la scrivi in avanti, e il disegno era già fatto. È l’errore numero uno che faranno fra cinque minuti.

Fase 3 · Officina 1 (20–45)

Venticinque minuti, e il compito è uno solo:

> »Nimm DEIN Bild von letzter Woche. Schreib oben drei Variablen hin. Und dann ersetz die Zahlen.«

Non un disegno nuovo. Il loro, quello di giovedì.

Come si fa, in pratica — scrivilo alla lavagna e lascialo lì:

```
1. Oben:    groesse = 100
            start_x = -100
            start_y = -100

2. Am Anfang:  penup()
               goto(start_x, start_y)
               pendown()
               setheading(0)

3. Dann:    forward(100)      ->   forward(groesse)
            forward(50)       ->   forward(groesse * 0.5)
            forward(200)      ->   forward(groesse * 2)
```

La prova che ha funzionato la fanno da soli, e falla annunciare: cambiano `groesse` e premono Play. Se il disegno si ingrandisce tutto insieme, è a posto. Se si ingrandisce a metà, è rimasto un numero fisso da qualche parte.

> »Wenn nur die Hälfte größer wird: irgendwo steht noch eine alte Zahl. Such sie.«

È un'autoverifica che non richiede te, e per una volta l'errore si vede invece di essere scritto in rosso.

Fase 4 · Il ciclo (45–60)

Lo steccato del compagno

Apri **il file dello steccato** fatto da un ragazzo giovedì. Chiedigli il permesso davanti a tutti.

Scorri le righe e chiedi, neutro — **non** «potevi farlo meglio», che suona come un rimprovero e lo chiude:

> »Wie oft steht hier das Gleiche?«

Contano loro. Sette, otto volte.

Poi apri `zaun_kurz.py` e lo esegui: stesso steccato. Poi fai vedere il codice: **sette righe per tutte le latte.**

```
        for i in range(8):
            left(90)
            forward(hoehe)
            backward(hoehe)
            right(90)
            penup()
            forward(abstand)
            pendown()
```

Due cose da dire, e basta:

`for i in range(8):`	«fai otto volte quello che viene dopo»
I due punti e il rientro	**tutto quello che è spostato a destra sta dentro il ciclo.** Quello che non è spostato, no

La scala: la prova che è la stessa cosa

Prima di passare oltre, una verifica che chiude ogni dubbio. Apri `treppe.py` di giovedì scorso — ventiquattro righe — e `treppe_kurz.py`. Play tutte e due.

Lo stesso identico disegno. Le ho confrontate pixel per pixel: zero differenze.

```
        for i in range(stufen):
            forward(breite)
            left(90)
            forward(hoehe)
            right(90)
```

Poi cambi `stufen` da 5 a 12 e premi Play: dodici gradini. Con la versione lunga sarebbero ventotto righe da battere a mano.

> »Gleiches Bild. Vier Zeilen statt vierundzwanzig. Und jetzt zwölf Stufen, ohne eine einzige Zeile mehr.«

Qui si vedono le due cose di oggi insieme: il ciclo toglie la ripetizione, le variabili in cima decidono quanto e dove.

Il colpo che non si può fare a mano

`vieleck.py`, e cambi solo `ecken`:

- `ecken = 3` → un triangolo
- `ecken = 5` → un pentagono
- `ecken = 36` → **un cerchio**

> »Sechsunddreißig Seiten. Das würde keiner von euch von Hand tippen.«

Il perimetro resta sempre 600, quindi la figura resta della stessa grandezza e si vede solo cambiare la forma.

I due errori che faranno (e sono questi due)

Vale il metodo di giovedì: tre domande, e chi trova dice solo il numero di riga.

`fehler_7.py` — mancano i due punti.

```
line 7 · SyntaxError: expected ':'
```

`fehler_8.py` — il `right(90)` è fuori dal ciclo. **Nessun messaggio rosso**: esce una riga lunga invece di un quadrato.

> »Kein Fehler. Aber schaut, was gezeichnet wird. Was ist im Zyklus drin und was nicht?«

Fase 5 · Officina 2 (60–83)

Ventitré minuti, e si divide in due senza che tu debba organizzare niente:

- **Chi ha finito le variabili** prende il ciclo e accorcia una ripetizione nel proprio disegno.
- **Chi è ancora indietro** continua con le variabili. Va benissimo: la variabile è il contenuto principale di oggi.

Non obbligare nessuno a usare il ciclo oggi. Chi lo vede funzionare sullo schermo di un compagno ci arriva la settimana prossima.

Fase 6 · Conta e quaderno (83–90)

Due numeri sul quaderno, e sono la cosa che li gratifica di più perché non dipende dall'essere bravi a disegnare:

```
 Zeilen vorher:  42
Zeilen nachher: 19
```

In Thonny il numero di righe si legge in basso a destra.

E una riga: cosa hanno messo nella variabile `groesse`.

Se finiscono prima

Se finiscono in due o tre, mentre gli altri lavorano

Due case, una variabile. Copiano il blocco della casa due volte, cambiano solo `start_x` fra una e l'altra. Scoprono che con le variabili duplicare un pezzo costa due secondi — e incontrano da soli il problema che la seconda casa usa la stessa `groesse`.

La casa che si rimpicciolisce da sola. Tre case in fila, `groesse` × 1, × 0.7, × 0.5. Sembra prospettiva e costa tre righe.

Il foglio delle sfide per chi va forte: è in fondo a questo documento, quattro problemi veri con la matematica dentro. Daglielo stampato e lascialo lavorare da solo — non ha bisogno di te, ha bisogno di qualcosa di difficile.

Se finiscono quasi tutti, e restano dieci o quindici minuti

La gara del file più corto. Chi ha il programma con meno righe che disegna ancora la stessa cosa. Si legge il numero in basso a destra in Thonny, e si proietta il vincitore.

Il ciclo dentro il ciclo, solo come spettacolo. Al proiettore, non da copiare: `vieleck.py` dentro un altro ciclo che ruota di 10 gradi ogni volta. Esce una rosetta. »Nächste Woche.«

Se avanzano cinque minuti

Indovina la figura. Scrivi `ecken = 8` sul proiettore senza eseguire, e la classe dice che forma uscirà. Poi Play.

Gli inciampi

Cosa vedi	Cos'è	Cosa fare
`NameError: name 'groesse' is not defined`	La variabile è scritta dopo l'uso, o scritta in modo diverso	»Wo steht sie? Und wie heißt sie genau?« Maiuscole comprese
Metà disegno si ingrandisce, metà no	È rimasto un numero fisso	È l'autoverifica: fallo cercare da lui, non cercarlo tu
Cambia `groesse` in fondo al file e non succede niente	La variabile vale da dove è scritta in avanti	»Vorher oder nachher?«
`SyntaxError: expected ':'`	Mancano i due punti del ciclo	`fehler_7`, l'ha già visto
Il ciclo disegna una riga lunga	Il rientro: il `right` è fuori dal ciclo	`fehler_8`. »Was ist eingerückt und was nicht?«
`IndentationError`	Rientro incoerente, spazi e tabulazioni mischiati	In Thonny: tutto il blocco selezionato e Tab. Non farglielo sistemare a mano riga per riga
Il disegno sparisce dallo schermo	`start_x` o `start_y` troppo grandi	Lo schermo va da −450 a 450 circa. »Welche Zahl ist zu groß?«

Differenziazione

Chi è lentissimo a scrivere. Oggi guadagna più di tutti: con tre variabili in cima non deve più ribattere quindici numeri per spostare la casa. Se arriva solo a mettere `groesse` e a sostituire cinque `forward`, ha fatto la lezione.

Il ragazzo che va forte. Il foglio delle sfide, stampato, e lo lasci in pace. Ha fatto una casa centrata da solo e usa radici e Pitagora: quattro problemi che richiedono matematica vera gli servono più di qualsiasi spiegazione tua. Passa da lui una volta a metà ora e chiedi di farti vedere, non di spiegarti.

Chi si blocca e non chiama. Oggi c'è un sintomo visibile: se cambia `groesse` e il disegno non cambia, è fermo. Il giro dietro ai banchi lo vedi dallo schermo senza chiedere niente.

Ha funzionato?

- Almeno otto su undici hanno tre variabili in cima al proprio file.
- Qualcuno ha cambiato `groesse` davanti a te e il disegno è scalato tutto insieme.
- Almeno tre hanno un `for` nel proprio file.
- Qualcuno ha trovato da solo il «numero fisso rimasto» senza che glielo cercassi tu.
- Sul quaderno ci sono due numeri: righe prima, righe dopo.

Il foglio delle sfide · da stampare e consegnare

Questa pagina è in tedesco perché la dai in mano a lui. Quattro problemi, in ordine di difficoltà. Le soluzioni sono subito dopo: non dargliele.

HERAUSFORDERUNGEN · für wenn dir langweilig wird

Du brauchst dafür eine neue erste Zeile. Damit kann Python rechnen:

```
from math import sin, cos, sqrt, atan, radians, degrees
```

1 · Das Vieleck im Kreis

Zeichne ein Vieleck mit `ecken` Ecken, das **genau in einen Kreis mit Radius 150 passt** — und zwar für jede Eckenzahl, ohne dass du etwas von Hand anpasst.

Zeichne danach mit einer anderen Farbe `circle(150)` darüber. Wenn du es richtig gemacht hast, liegen alle Ecken **auf** dem Kreis.

> **Tipp:** die Seitenlänge ist `2 * radius * sin(radians(180 / ecken))`. Und damit es mittig steht: der Abstand vom Mittelpunkt zur Seitenmitte ist `radius * cos(radians(180 / ecken))`.

2 · Pythagoras

Zeichne ein rechtwinkliges Dreieck mit den Katheten `a = 240` und `b = 180`.

Die Hypotenuse darfst du **nicht abmessen und nicht raten**: der Computer rechnet sie aus. Schreib die Länge mit `write()` unter das Dreieck.

> **Tipp:** `c = sqrt(a * a + b * b)`. Für die Drehung brauchst du auch den Winkel: `degrees(atan(b / a))`.
>
> Kontrolle: bei 240 und 180 muss genau 300 herauskommen.

3 · Das Sternpolygon

Ein Stern mit fünf Zacken geht mit `right(144)`. Warum 144?

Die allgemeine Regel: bei `ecken` Punkten und `sprung` Schritten weiter ist die Drehung `360 * sprung / ecken`.

Probier `ecken = 7` mit `sprung = 2` und mit `sprung = 3`. Dann `ecken = 8`, `sprung = 3`.

> **Frage, auf die du selbst kommen sollst:** bei welchen Kombinationen wird der Stern **nicht** fertig, sondern hört nach ein paar Strichen auf? Was haben die Zahlen dann gemeinsam?

4 · Die Spirale

```
for i in range(120):
    forward(i * 2)
    right(90)
```

Das wird ein Quadrat, das nach außen wächst. Jetzt ändere die 90 in **89**.

Dann probier 120, 144, 61.

> **Frage:** warum dreht sich das ganze Bild, wenn der Winkel nur um ein Grad daneben liegt?

Le soluzioni delle sfide

Da tenere per te. Verificate eseguendole.

1 · Vieleck im Kreis

```
from turtle import *
from math import sin, cos, radians

ecken = 7
radius = 150

seite = 2 * radius * sin(radians(180 / ecken))
abstand_mitte = radius * cos(radians(180 / ecken))

penup()
goto(-seite / 2, -abstand_mitte)
pendown()
setheading(0)
pensize(3)
color("darkblue")
for i in range(ecken):
    forward(seite)
    left(360 / ecken)

penup()
goto(0, -radius)
pendown()
setheading(0)
color("red")
circle(radius)

done()
```

2 · Pythagoras

```
from turtle import *
from math import sqrt, atan, degrees

a = 240
b = 180

c = sqrt(a * a + b * b)
winkel = degrees(atan(b / a))

penup()
goto(-a / 2, -b / 2)
pendown()
setheading(0)
pensize(3)
color("darkgreen")

forward(a)
left(90)
forward(b)
left(180 - winkel)
forward(c)
left(90 + winkel)

penup()
goto(-a / 2, -b / 2 - 40)
color("black")
write("c = " + str(round(c, 1)), font=("Arial", 18, "bold"))

done()
```

Con 240 e 180 esce esattamente 300: è un triangolo 3-4-5 moltiplicato per 60. Il numero tondo è voluto, serve a dargli la conferma che il conto è giusto.

3 · Sternpolygon

```
from turtle import *

ecken = 7
sprung = 3

penup()
goto(-100, -120)
pendown()
setheading(0)
pensize(3)
color("darkred")
for i in range(ecken):
    forward(220)
    right(360 * sprung / ecken)

done()
```

La risposta alla domanda: se `ecken` e `sprung` hanno un divisore comune, la figura si chiude prima e non passa per tutti i punti. Con 8 e 2 vengono due quadrati sovrapposti; con 8 e 3 viene la stella intera. È il massimo comun divisore, e a un ragazzo che fa già Pitagora si può dire così.

4 · Spirale

```
from turtle import *

winkel = 89
schritte = 120

speed(0)
pensize(2)
color("purple")
penup()
goto(0, 0)
pendown()
for i in range(schritte):
    forward(i * 2)
    right(winkel)

done()
```

La risposta alla domanda: con 90 gradi la tartaruga torna esattamente nella direzione di partenza ogni quattro tratti, quindi la figura resta dritta. Con 89 ogni giro resta indietro di un grado, e l'intera figura ruota lentamente: dopo 360 tratti ha fatto un giro completo. È lo stesso motivo per cui le lancette di un orologio formano quelle figure.

Allegato · il codice

Da copiare nella cartella `python_kurs` di ogni macchina il mercoledì. I due file rotti nella sottocartella `fehler`, accanto ai sei di giovedì scorso.

`haus_variabel.py`

```
# ============================================================
#  HAUS MIT VARIABLEN
#
#  Oben stehen drei Zahlen. Alles andere rechnet sich daraus aus.
#  Ändere NUR die drei Zahlen oben und drück Play.
# ============================================================

from turtle import *

speed(0)          # 0 = sofort, 1 = langsam zum Zuschauen

# ============================================================
#  HIER WIRD ALLES EINGESTELLT
# ============================================================

groesse = 120      # <-- wie groß das Haus ist
start_x = -60      # <-- wo es steht: links / rechts
start_y = -120     # <-- wo es steht: unten / oben

# ============================================================
```

```
        # --- An den Start ---
        penup()
        goto(start_x, start_y)
        pendown()
        setheading(0)

        # --- Die Wände ---
        color("black", "firebrick")
        begin_fill()
        forward(groesse)
        left(90)
        forward(groesse)
        left(90)
        forward(groesse)
        left(90)
        forward(groesse)
        left(90)
        end_fill()

        # --- Nach oben zum Dach, ohne zu zeichnen ---
        penup()
        left(90)
        forward(groesse)
        right(90)
        pendown()

        # --- Das Dach ---
        color("black", "sienna")
        begin_fill()
        left(60)
        forward(groesse)
        right(120)
        forward(groesse)
        right(120)
        forward(groesse)
        end_fill()

        # --- Die Tür ---
        penup()
        goto(start_x + groesse * 0.4, start_y)
        pendown()
        setheading(90)
        color("black", "saddlebrown")
        begin_fill()
        forward(groesse * 0.45)
        right(90)
        forward(groesse * 0.2)
        right(90)
        forward(groesse * 0.45)
        end_fill()

        # --- Das Fenster ---
        penup()
        goto(start_x + groesse * 0.4, start_y + groesse * 0.6)
        pendown()
        setheading(0)
        color("black", "lightblue")
        begin_fill()
        forward(groesse * 0.2)
        left(90)
        forward(groesse * 0.2)
        left(90)
        forward(groesse * 0.2)
        left(90)
        forward(groesse * 0.2)
        left(90)
        end_fill()

        done()
```

treppe_kurz.py

```python
# ============================================================
#  TREPPE MIT SCHLEIFE
#
#  Letzte Woche: fünf Stufen, fünf Mal das Gleiche getippt.
#  Heute: einmal getippt, fünf Mal gemacht.
#
#  Und oben stehen die Zahlen, die du ändern darfst.
# ============================================================

from turtle import *

speed(0)          # 0 = sofort, 1 = langsam zum Zuschauen
pensize(5)
color("saddlebrown")

# ===== EINSTELLUNGEN =====
stufen = 5         # <-- wie viele Stufen
breite = 40        # <-- wie breit eine Stufe
hoehe = 40         # <-- wie hoch eine Stufe
start_x = -120     # <-- wo die Treppe anfängt
start_y = -120

# --- An den Start ---
penup()
goto(start_x, start_y)
pendown()
setheading(0)

# --- Die Stufen ---
for i in range(stufen):
    forward(breite)
    left(90)
    forward(hoehe)
    right(90)

done()
```

zaun_kurz.py

```
# ============================================================
#  ZAUN MIT SCHLEIFE
#
#  Acht Latten, und trotzdem nur sechs Zeilen für alle acht.
#  Die Schleife macht das Gleiche mehrmals.
# ============================================================

from turtle import *

speed(0)
pensize(6)
color("saddlebrown")

# ===== EINSTELLUNGEN =====
latten = 8          # <-- wie viele Latten
abstand = 30        # <-- wie weit auseinander
hoehe = 90          # <-- wie hoch

# --- An den Start ---
penup()
goto(-150, -80)
pendown()
setheading(0)

# --- Die Latten ---
for i in range(latten):
    left(90)
    forward(hoehe)
    backward(hoehe)
    right(90)
    penup()
    forward(abstand)
    pendown()

# --- Die zwei Querlatten ---
penup()
goto(-150, -80 + hoehe * 0.3)
pendown()
setheading(0)
forward(abstand * (latten - 1))

penup()
goto(-150, -80 + hoehe * 0.7)
pendown()
forward(abstand * (latten - 1))

done()
```

vieleck.py

```
# ============================================================
#  VIELECK – eine Zahl, jede Form
#
#  Ändere nur "ecken" und drück Play:
#  3 = Dreieck, 4 = Quadrat, 5 = Fünfeck, 36 = fast ein Kreis.
#
#  Der ganze Umfang bleibt immer 600 lang.
# ============================================================

from turtle import *

speed(0)
pensize(3)
color("purple")

ecken = 36          # <-- probier 3, 4, 5, 8, 12, 36

for i in range(ecken):
    forward(600 / ecken)
    right(360 / ecken)

done()
```

fehler/fehler_7.py

```
        # FEHLER 7 – Findest du ihn?
        from turtle import *
        speed(0)
        pensize(5)
        color("blue")

        for i in range(4)
            forward(100)
            right(90)

        done()
```

fehler/fehler_8.py

```
        # FEHLER 8 – Kein roter Text. Trotzdem ist es falsch.
        # Es soll ein Quadrat werden.
        from turtle import *
        speed(0)
        pensize(5)
        color("green")

        for i in range(4):
            forward(100)
        right(90)

        done()
```